Business Entity Resolution at Scale: Approach of Team ARCA

Amazon ML Challenge 2026 | Team **ARCA**: Eklavya Singh, Kamran Ansari, Ayush Singh

Public leaderboard (macro F0.5)	**0.9869**, rank 346 among 18,000+ teams
Validation (held-out 20% of training entities, exact metric)	**0.9891**
Candidates per Source-1 entity	**48.4**, recall of true pairs 98.3%
Data processed per split	~1.7-2.2M Source-1 entities, ~10M Source-2/3 records
Hardware	one 8 GB laptop + free Kaggle T4 GPUs
Models	LightGBM + three open cross-encoders (MIT / Apache-2.0, 33M-278M parameters)

This document covers the four points requested: methodology (section 1), candidate generation (section 2), model architecture and feature engineering (section 3) and other relevant information (section 4). The code that produced the submission is in `src/`; `run_pipeline.sh` runs it end to end.

---

1. Methodology

1.1 How we framed the problem

Three properties of the task decided most of our design:

1. **The metric is F0.5 per Source-1 entity, averaged over all entities.** Precision counts more than recall, a wrong match costs about three times as much as a missed one, and every entity counts equally. An entity with no true match (5.6% of them) scores 1 only if we predict nothing for it. So the last step is not a threshold but a per-entity decision about how many matches to output.
2. **Every Source-2/3 record belongs to at most one Source-1 entity.** We verified this on all 7.6M training matches. It turns the problem into an assignment problem: records compete for one owner.
3. **The data contains planted look-alikes and a shifted test set.** Training has copies with typos, abbreviations, reordered words, websites as names, ten Indic scripts and missing addresses, but also hard negatives: records that copy a business's building with one name word changed, or its name with a nearby house number. The test adds a country never seen in training (France, 15% of the entities), uses smaller splits and contains more of these look-alikes (5.8 records per entity against 4.7 in training, with the same number of true matches).

1.2 Pipeline

```
raw TSV -> normalise -> blocking (15 key families) -> ~78 pair features
   |
   v
stage 1: LightGBM, two out-of-fold models -> p1
   |
   v
stage 2: context, group evidence, deviation support, word track records
   |
   +-- 3 cross-encoders (MiniLM, multilingual MiniLM, multilingual-e5), folds 1-2
   v
final stacker: LightGBM on 148 features (+ copy counts, raw spelling)
   |
   v
test-shift corrections -> exclusive assignment -> expected-F0.5 choice per entity
   |
   v
second pass for unassigned records -> matching_results.tsv, candidate_pairs.tsv
```

1.3 Validation that mirrors the test

A good validation set was worth more than any single model, so we built it carefully:

- **Held-out entities, not held-out pairs.** Source-1 entities are split into five folds by a hash of their id; fold 0 (20%) never takes part in training.
- **The full record universe.** Validation candidates are generated against all 10M training records, so a held-out entity faces the same look-alikes and competitors as a test entity.
- **An honest chain of out-of-fold scores.** Stage 1 is trained twice (folds 1-2 and folds 3-4) so that every training pair is scored by the model that did not see it; the cross-encoders are trained on folds 1-2 only; the stacker is trained on folds 3-4 and validated on fold 0. No score that feeds a later stage comes from a model that saw that pair.
- **The exact metric.** `metric.py` re-implements the challenge metric (per entity, singletons included) and was checked against the README example and an all-empty submission.
- **Test-weighted validation.** An adversarial classifier (validation pairs against test pairs) gives each validation entity a weight, so decision rules are chosen on a validation set that looks like the test.

1.4 Results

Public leaderboard F0.5	**0.9869**
Validation F0.5 (fold 0, exact metric)	**0.9891**
Candidate recall on validation	98.3% (US 98.6%, India 97.8%) at 48.4 candidates per entity
Test output	5,793,569 matches for 1,732,544 entities; 5.9% of entities predicted without copies

The leaderboard score sits 0.2 points below validation. Most of the work in section 4 (the feature-scale fix, the France rules, the decoy-density correction) exists to keep that gap small, because the test differs from training in ways a validation score alone cannot reveal.

---

2. Candidate Generation / Blocking Strategy

Comparing every Source-1 entity with every record would mean about 2 x 10^13 pairs per split. Our blocking produces 48 candidates per entity while keeping 98.3% of the true pairs, and its cost grows roughly linearly with the data.

2.1 Normalisation

Blocking keys are only as good as the tokens they are built from. `normalize.py` turns every name and address into lower-case ASCII tokens:

- **Legal forms** (LLC / L.L.C. / Inc / Pvt Ltd / Private Limited / SARL / SAS ...) are mapped to canonical tokens and kept apart (`n_legal`), so "Acme LLC" and "Acme L.L.C." share a name, while the legal form stays available as a feature.
- **Digit-for-letter typos** inside words are undone ("fu1ler" -> "fuller", "5anjeev" -> "sanjeev"); pure numbers are kept.
- **Ten Indic scripts** are mapped to English tokens with a dictionary *learned from the training pairs*: for true pairs whose record is written in, say, Devanagari, we count which English words of the matching entity co-occur with each native word, and keep the best one when it also looks alike after transliteration. "Sunrise Foods" written in Devanagari becomes "sunrise foods" without any external resource.
- **Aliases and websites**: "X dba Y" / "X a/k/a Y" give a second name; a website used as a name ("wilfordhancock.com") is reduced to its stem ("wilfordhancock").
- **Addresses**: street-type abbreviations unified (Street / St / Str, Rue / R.), numbers extracted in order with leading zeros removed, "N°", "#", "No." handled, so "N° 0050 Rue X" and "50 R. X" agree.

2.2 Fifteen key families

Each record emits short keys. Every key starts with the country, so records are only compared within a country, and country is treated as an open set of values (France needed no special code here).

family	key	catches
1	pair of name tokens	most copies
2	single name token	short names
3	name token + house number	same business, noisy name
4	house number + address word	name rewritten or aliased
5	two adjacent address words	records without a number
6	name without spaces	"SunriseFoods" vs "Sunrise Foods", websites
7	first 7 characters of the space-free name	truncated names, typos at the end
8	consonant skeleton of two name tokens	vowel typos ("Mrktng")
9	name token + address word	name partly changed
10	house number + two adjacent address words	busy streets
11, 12	loose name-token pair / single token	records without an address
13	full name + house number	very common names made selective
14	full name + address word	very common names made selective
15	full name (loose)	records without an address

2.3 Selectivity and scoring

A key is only used if it is rare: at most 15 Source-1 entities carry it and it creates at most 500 entity-record pairs. The loose families (11, 12, 15) are emitted by every entity but only by records *without* an address, and get wider limits (150 / 5,000), because for those records the name is the only evidence. Each shared key adds a weight

```
w = ln(1 + 1000 / (n1 * n2))
```

to the pair, where n1 and n2 count the entities and records carrying the key, so a key shared by one entity and one record weighs far more than a common one. Each entity keeps its 50 best pairs by this score, plus the 15 best pairs by name keys alone, so that a busy building cannot crowd out a record whose name is a strong match.

2.4 Built to scale

- Key frequencies are counted in slices of the key hash, and pairs are scored per bucket of entities, so no step ever holds all keys or all pairs in memory. The whole pipeline runs on an 8 GB laptop for ~10M records per split.
- Only records that share a rare key are ever compared, so the work grows roughly linearly with the number of records, not with its square.
- Key families and caps are plain constants: a smaller candidate set is one parameter away. Measured on training: 48 candidates per entity keep 98.3% of the true pairs, 39.5 keep 98.2%, 30.5 keep 98.0%.

2.5 Second pass: retrieval from the record's side

A detailed look at the true pairs blocking still missed showed one dominant pattern: the record keeps only the first chunk of the address ("A-45, CHENNAI, Tamil Nadu", "4/71, Coimbatore, TN") and its name is a typo, a website or an Indic-script spelling. Seen from the entity, such a record is weak; seen from the record, the entity is often its only plausible owner. The second pass (`record_side.py`) therefore starts from records left unassigned by the main model and looks for entities sharing a rare key with them:

- **house codes** that keep their letters or second number ("a.45", "46.1106"): the plain number 45 is far too common in Chennai, "A-45" is not;
- **name word x town**, for typo, website and Indic-script names at a cut-down address;
- **pairs of address words**, for US copies whose house number was changed or dropped.

A dedicated LightGBM, trained on training labels, scores these pairs (character similarity separates typos from planted word changes). Picks above 0.85 are added (France 0.95: there, a real-word name at the same address is more often a neighbouring business). On a 35% sample of the validation entities this pass adds 976 pairs at 95.6% precision (+0.0002 F0.5 on that sample); on the test it recovers 11,383 matches. These pairs are added to

`candidate_pairs.tsv`, so every match is a candidate.

2.6 The submitted candidate set

candidate ids	83,879,604 for 1,732,544 entities (48.4 per entity)
recall on validation	98.3% (US 98.6%, India 97.8%)
matches that are candidates	100%

---

3. Model Architecture and Feature Engineering

3.1 Pair features (~78, `features.py`)

group	features
name similarity	ratio, token-set, token-sort, partial ratio and Jaro-Winkler on the cleaned name, the name with legal form, the space-free name and the dba alias; acronym match; first / last token equal
what differs	IDF-weighted token overlap in both directions; IDF of the rarest token present on one side only
legal form	agreement / conflict of the canonical legal forms
address	the same scorers on the cleaned address; IDF overlap; rarest unmatched address token
numbers	house number equal / absolute difference; shared numbers; smallest gap between numbers found on one side only (302 vs 306 -> 4, the signature of a neighbour-unit decoy); is the entity's number anywhere in the record
popularity	how many entities and records carry the same name (chains: the address must decide)
blocking	relative blocking score, number of shared keys, rank of the pair for the entity and for the record

Word rarity (IDF) is computed once on train and test text together, so a word has the same weight in both splits. Scale-dependent quantities are only used in relative form.

3.2 Stage 1

LightGBM on the pair features, trained twice on disjoint halves of the training folds. Its score p1 feeds every later feature that needs a first opinion.

3.3 Stage 2: what a single pair cannot see (`stage2.py`)

- **Context**: rank and gap of the pair's p1 among the entity's candidates and among the record's candidate entities (records compete for one owner).
- **Group evidence**: is the record a near twin (name or address) of the records the entity is already confidently matched to? Copies of one business look like each other.
- **Deviation support**: do the record's differences from the entity (an extra word, another number) recur among the entity's confident copies? Real copies tend to share their variations, look-alikes do not.
- **Word track records**: for each word a record *adds* to or *drops* from the entity's name or address, how often that change meant a true match in training (smoothed log-odds, computed out of fold). This is how the model learns that adding "Holdings" or "International" as the only change is a planted decoy (0% true), while adding "Services" or "Center" is a harmless variation (94-98% true).

3.4 Three cross-encoders

	model	licence	size	reads	scores
xenc	cross-encoder/ms-marco-MiniLM-L-12-v2	Apache-2.0	33M	normalised text	all stage-2 pairs
xenc2	cross-encoder/mmarco-mMiniLMv2-L12-H384-v1	Apache-2.0	118M	raw text, any script	all stage-2 pairs
xenc3	intfloat/multilingual-e5-base	MIT	278M	raw text, any script	3.0M uncertain pairs

Each reads "name | address" of both sides as one sequence and is fine-tuned only on training pairs of folds 1-2, so its scores are out of sample everywhere they are used. The multilingual models read Devanagari, Tamil or

French as written; xenc3 only scores pairs where either of the first two is unsure (score between 0.01 and 0.99), which keeps GPU time small. For each score the stacker also gets its view of the entity's group: rank, best score, gap and count above 0.5.

3.5 Final stacker (`stacker.py`)

LightGBM on 148 features: the stage-2 features, the three cross-encoder scores with their group views, and two feature families aimed at entities that share a name:

- **Per-source copy counts**: how many confident copies the entity already has in the record's source, and the same for the record's best competing entity.
- **Raw spelling**: normalisation maps "Moque L.L.C.", "Moque LLC" and "Moque" to the same tokens, but each copy inherits the raw spelling of its own entity. We compare names folded only for case and accents (punctuation and legal-form spelling kept), plus the rank and gap of that similarity among the record's candidates and an exact comparison of the raw legal-form signature.

Training for the unknown. French words never occur in training, so their word track records are missing on the test. A model that always saw them in training has no good answer when they disappear. We hide the added-word track record on 30% of the training rows that add a word, so the model also learns to judge added words from the cross-encoders, which read the words themselves. France is then scored by a second stacker trained without the English-only cross-encoder, which reads French words as unknown fragments.

3.6 Decision layer (`decide.py`, `final_assemble.py`)

1. **Exclusive assignment**: each record is kept only for its most probable entity.
2. **Expected-F0.5 selection**: for each entity, keep the top k candidates that maximise the expected F0.5,

   ```
   E[F | k] = 1.25 * sum_{i<=k} p_i / (0.25 * sum_all p + k),    E[F | 0] = prod(1 - p_i)
   ```

   The second term is the probability that the entity has no match at all, so the rule outputs nothing when that is the better bet, which matters for the 5.6% of entities without copies. A calibration factor (0.8) was chosen on the test-weighted validation.

---

4. Other Relevant Information: What Made the Difference

4.1 Finding a bug that only existed on the test

Without this step the leaderboard trailed validation by 0.3 points, and no validation experiment could explain it. We compared, without labels, how many matches the model accepted at each confidence level per entity, test against validation, country by country. India matched validation almost exactly, the US had twice as many uncertain matches, France three to seven times as many. The US excess was almost entirely one kind of pair: a record without an address whose name matched an entity exactly. The test did not contain more such records, so the model had to be judging the same records differently.

The cause was a name-popularity feature defined as a rate per 100,000 entities of the split. The test split is half the size of the training split for the US (663K against 1.32M entities), so a name used by one test entity got the value that a name shared by two entities has in training, and by four in France. The model read it as "another company has this name" and doubted every such copy. Rescaling the test values to the training scale (`freq_fix.py`) restored the first-stage scores of these copies from 0.58 to 0.975 (US) and from 0.34 to 0.966 (France), the same as validation, while India, whose split sizes are similar, stayed unchanged at 0.973. Together with the French rules below, this step adds 0.0008 to the leaderboard score.

4.2 A country with no labels: learning France from how the data was built

France has no training labels, so we could not train on it. Instead we measured the generator's habits on training labels and looked for the same habits, written in French, in the unlabelled test data:

- In training, a record that *adds* "Group", "Holdings", "India" or "International" to a name is a planted decoy

(0% true). The French test shows the same pattern for "Groupe", "Développement", "France", "Holding", "International", "Participations" and "Distribution". These are scaled down.
- In training, a copy at the same address that replaces a word with "Center", "Services" or "Partners" is a true copy 99.4-99.9% of the time. The French equivalents ("& Fils", "& Associés", "Cie", "Services", "Frères") are unknown to the model, so such pairs are scaled up.
- Swaps of one common French category word for another ("Club" -> "Amicale", "École" -> "Comité") and records with the same name and house number on a *different street* behave like planted decoys and are scaled down.

To check these rules without labels we used a simple signal: planted decoys are attached to random entities, including entities with no copies, so a decoy is far more often the *only* match of its entity. Normal French matches are an entity's only match 2.4% of the time; the vocabulary swaps 6.0%, the other-street matches 10.2%. The same signal showed that "Centre" and "Compagnie" act as category words in France, not as copy suffixes (3.5-6.2% against 1.9-2.4%), so they were moved to the decoy side.

4.3 Correcting for more decoys in the test

The test has more look-alikes per entity than training, and they are concentrated in known categories. Pairs with the same name and a house number changed by 1-99 are 1.8x (US), 1.4x (India) and 2.3x (France) more frequent among test candidates than validation candidates, while true pairs of that kind are not. By Bayes' rule, the odds of such a pair being true are multiplied by the ratio of the two decoy densities (0.47-0.65 for US and India, 0.05-0.33 for France). This uses only counts from the unlabelled test file.

4.4 Companies that share a name

39% of the entities share their cleaned name with another entity. With an address this is easy; without one, the evidence is subtle. We added three kinds of evidence that normalisation would otherwise erase or ignore: the raw spelling of the name and its legal form, per-source copy counts (an entity that already has its copies in one source is a less likely owner of another), and the record-side competition features. The remaining errors on validation are mostly records without an address whose name belongs to several entities with *identical* raw names: the data holds no information to tell them apart, and our rule correctly declines to guess when a wrong guess costs more than a right one gains.

4.5 Engineering under tight constraints

- **One 8 GB laptop** for ~10M records per split: every heavy step streams Parquet parts, partitions keys by hash and entities by bucket, and runs under a small guard (`guard.sh`) that stops a step before it exhausts memory or disk.
- **Free Kaggle T4 GPUs** for the cross-encoders: data is packed into compact Parquet packages, and the largest model scores only the uncertain band (3.0M of 84M pairs).
- **Integer keys** (source x 10^10 + id) instead of strings for all joins; similarity scores stored as one byte.

4.6 Compliance and reproducibility

- Only the provided training and test files are used. No external data, gazetteers, geocoding, APIs or language-model services. The Indic dictionary is learned from training pairs.
- All models are open-weight: LightGBM, MiniLM-L12 (Apache-2.0), mMiniLMv2 (Apache-2.0), multilingual-e5-base (MIT), all far below 8B parameters, fine-tuned on the provided training pairs and run offline.
- No test labels were used. Test-side corrections rely only on unlabelled test statistics (counts and densities), which the organisers allowed.
- `run_pipeline.sh` runs the pipeline end to end. With the submitted code, retraining the final stacker and the second-pass model reproduces both model files byte for byte; re-running the final steps reproduces the submitted matches up to 218 pairs (0.013% of entities), caused by a tie between equally similar copies in one group feature whose order depends on threads (documented in the README). GPU fine-tuning is not bit-exact, so a full re-run from scratch matches up to such small differences.

4.7 What still limits the score

On validation the remaining loss (1.1 points) splits into: records without an address whose name several entities share (0.64), true copies the blocking never proposed (0.31), wrong matches (0.10) and true pairs below the rule (0.07). The first group is largely undecidable from the data, which is why we invested in blocking recall, the second pass and the test-shift corrections, where the score could still move.